

第一次课后作业 (概述+线性表)

题量: 1 满分: 100

智能分析

94分

作答时间: 2025-02-19 20:10 至 2025-03-04 23:00

一. 简答题 (共1题, 100分)

1. (简答题)

写出下列程序段平均情况下时间代价的 Θ 表示式, 要求给出分析过程和结论。假设所有变量类型都为int:

AI讲解

(a)

```
sum = 0;
for (i = 1; i <= n; i*=2)
    for (j = 1; j <= n; j++)
        sum ++;
```

(b)假设数组A中含有n个元素, 函数Random花的时间是常数值, sort需要执行 $n\log n$ 步。

```
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++)
        A[i] = Random(n);
    sort(A, n);
}
```

(c)假设数组A中元素为从0到n-1的任意一个排列。

```
sum3 = 0;
for (i = 0; i < n; i++)
    for (j = 0; A[j] != i; j++)
        sum3 ++;
```

(d)

```
sum = 0;
if (EVEN(n))
    for (i = 0; i < n; i++)
        sum ++;
else
    sum = sum + n;
```

2. 算法分析题，阅读以下代码：

```
int a[100];
Fun(int a[], int n)
{
    for(int i=1; i<=n; ++i)
    {
        cin>>a[i];
    }
    int K=1;
    for(int i=1; i<=n; ++i)
    {
        if(i > 1 && a[i] < a[i - 1])
            K = i;
        while (K < n && a[i] >= a[K + 1])
            ++ K;
        cout<< K;
    }
}
```

若输入的a数组是一个严格单调递增的数列，分析此程序的时间复杂度。

3. 分析下列程序段的时间复杂度（要求给出分析过程和结论）：

```
int fact(int n){
    int res=n;
    if(n>1) res=res*fact(n/2);
    return res;
}
```

4. 请分析算法1-12的时间复杂度（要求给出分析过程和结论）：

算法1-12: BinarySearch(array, x)

输入：整数数组 *array*，待查找的整数 *x*

输出：如果 *x* 在 *array* 中，输出其对应的下标；否则，输出 NIL

```
1.  n ← array.size
2.  left ← 0    // 执行一次
3.  right ← n-1 // 执行一次
4.  while left ≤ right do
5.    | middle ← (left+right)/2
6.    | if x=array[middle] then
7.      | | return middle
8.    | else if x<array[middle] then
9.      | | right ← middle - 1
10.   | else // x>array[middle]
11.     | | left ← middle + 1
12.   | end
```

5.给定顺序表 $L=(a_1, a_2, \dots, a_n)$,请设计一个时间和空间上尽可能高效的算法,将该表循环右移指定的 m 位。例如,将(1,2,5,7,3,4,6,8)循环右移3位($m=3$)后的结果是(4,6,8,1,2,5,7,3)。

(1) 给出算法的基本设计思想。

(2) 根据设计思想,采用C/C++语言或者伪代码描述算法,关键之处给出注释。

(3) 分析算法的时间复杂度。

6.如果链表采用带空头结点的方式实现,请修改算法2-7(单链表插入)和算法2-8(单链表删除),实现相应的插入和删除操作。

(1) 给出算法的基本设计思想。

(2) 根据设计思想,采用C/C++语言或者伪代码描述算法,关键之处给出注释。

(3) 分析算法的时间复杂度。

注意:课后作业必须在作业本上完成,需要有完整的计算过程。请于截止时间之前,在系统提交答案照片,并且每页纸上需要写上姓名。

1.

(a) $sum = 0$ $\oplus (1)$
 for ($i = 1; i \leq n; i \times 2$) 执行 $\log_2 n$ 次
 for ($j = 1; j \leq n; j++$) 执行 n 次
 $sum++$ $\oplus (1)$

故时间复杂度为 $\oplus (n \log n)$

(b) for ($i = 0; i < n; i++$) { 执行 n 次
 for ($j = 0; j < n; j++$) 执行 n 次
 $A[i] = \text{Random}()$
 $\text{sort}(A[i]);$ 执行 $n \log n$ 次
 } $\oplus (n \log n)$

故时间复杂度为 $\oplus (n^2 \log n)$

(c) $sum = 0$
 for ($i = 0; i < n; i++$) 执行 n 次
 for ($j = 0; A[j] != i; j++$); 最坏情况下执行 n 次.
 $sum++$;

$\therefore$ 时间复杂度为 $\oplus (n^2)$

(d) $sum = 0$
 if ($\text{EVEN}(n)$) 常数时间操作
 for ($i = 0; i < n; i++$); 执行 n 次
 $sum++$ $(n \text{ 为偶数})$
 else.

$\therefore$ 时间复杂度为 $\oplus (n)$

2. for (int $i = 1; i \leq n; i++$) 执行 n 次
 { $\text{cin} >> a[i];$
 }
 int $k = 1$

for (int $i = 1; i \leq n; i++$) { 执行 n 次
 if ($i \geq 1 \ \&\& \ a[i] < a[i+1]$) 常数时间操作
 $k = i$
 while ($k \leq n \ \&\& \ a[i] > a[k+1]$) 最多执行 n 次
 $i++$
 $\text{cout} << k;$
 }

$\therefore$ 时间复杂度为 $O(n) + O(n \cdot n) = O(n^2)$.

3. int fact(int n) {
 int $res = n$;
 if ($n > 1$) $res = res * \text{fact}(\frac{n}{2})$
 return res ;

递归: 每次将 n 除以 2 至小于等于 1. 递归 $\log_2 n$ 次
 故时间复杂度为 $O(\log n)$

4. $\left. \begin{array}{l} n \leftarrow \text{array.size} \\ \text{left} \leftarrow 0 \\ \text{right} \leftarrow n-1 \end{array} \right\}$ 为常数时间操作 $O(1)$.

while 循环二分查找 迭代次数 $\lg_2 n$

判断 为常数时间操作 $O(1)$.

$\therefore$ 时间复杂度为 $O(\log n)$.

5.

(1) 算法设计思想

① 计算有效右移步数 $m = m \% n$

② 将顺序表的后 m 个元素提取到临时数组中

③ 将顺序表前 $n-m$ 个元素右移 m 位

④ 将临时数组的 m 个元素插入顺序表前 m 个位置

(2) 代码.

```
void moveRight(vector<int>&arr, int m)
```

```
{ int n = arr.size();
```

```
    m = m % n    // 右移步数.
```

```
    vector<int> temp(arr.end()-m, arr.end()); // 将后  $m$  个元素保存到临时数组
```

```
    for(int i=n-1; i>=m; --i)
```

```
    { arr[i] = arr[i-m]; // 将前  $n-m$  个元素右移  $m$  位.
```

```
    }
```

```
    for(int i=0; i<m; ++i)
```

```
    { arr[i] = temp[i]; // 将后  $m$  个元素放在数组前  $m$  个位置
```

```
    }
```

```
}
```

(3) 时间复杂度: 提取后 m 个元素: $O(m)$. 右移 $n-m$ 个元素: $O(n-m)$

插入 m 个元素: $O(m)$

综上: 整体时间复杂度为 $O(n)$

算法 2-7 (单链表插入)

(1) 算法设计思想

- ① 带头节点的链表的头节点只是占位符不存储有效数据, 插入操作从头节点开始
- ② 遍历链表找到第 $i-1$ 个节点, 即插入点的前驱节点
- ③ 创建新节点, 将数据 x 插入链表, 新节点 $next$ 指针指向目标位置原节点, 前驱节点的 $next$ 指向新节点
- ④ 若目标位置超出链表长度, 退出

(2) 代码

```
void insertNode(LinkList *head, int x, int i){
```

```
    LNode *p = *head; // 从头节点开始, 头节点为头
```

```
    int counter = 0; // 计数, 找到第  $i-1$  个节点
```

```
    while (p != NULL && counter < i-1){
```

```
        p = p->next; // 遍历链表, 找到第  $i-1$  个节点
        counter++; }
```

```
    if (p != NULL) {
```

```
        LNode *new_node = new LNode; // 创建新节点
```

```
        new_node->data = x
```

```
        new_node->next = p->next;
```

// 新节点指向 p 的下一个节点

```
        p->next = new_node;
```

// p 的下一个节点指向新节点

```
    } else { cout << "位置不合法" << endl; }
```

(3) 时间复杂度:

① 找到第 $i-1$ 个节点, 必须遍历链表, 最坏情况遍历整个链表, 时间复杂度 $O(n)$

② 创建新节点, 为常设时间操作 $O(1)$

③ 综上, 总的时间复杂度为 $O(n)$

6. 算法 2-8 (单链表删除)

1. 算法设计思想

- ① 从空的头节点开始遍历链表.
- ② 查找第 i 个节点
- ③ 找到第 i 个节点后, 将要删除节点的前驱节点的 `next` 指针指向第 i 个节点的后继节点, 这样跳过待删除节点, 从而删除目标节点.
- ④ 释放删除节点的内存
- ⑤ 若找不到目标位置 i 返回 `false` 表示删除失败

2. 代码

```
bool deleteNode(Linklist *head, int i){  
    LNode *pre = *head; // 从头节点开始, 头节点是空节点  
    int counter = 0; // 计数器, 用来定位第  $i$  个节点  
    while(pre->next != NULL && counter < i-1){  
        pre = pre->next // 遍历链表, 找到第  $i$  个节点.  
        counter++;  
    }  
    if(pre->next != NULL){  
        LNode *temp = pre->next; // 临时指针指向待删除节点.  
        pre->next = pre->next->next; // 删除第  $i$  个节点  
        delete temp; // 释放内存  
        return true;  
    }  
    return false // 未找到合适位置返回 false
```

3. 时间复杂度

① 查找目标节点要遍历链表, 最坏情况下 $O(n)$.

② 删除操作为常数时间 $O(1)$.

综上, 时间复杂度为 $O(n)$.